

EDS THEORY ASSIGNMENT 2

Atharva Madan Jadhav

202501040350 CS3 – C35

Dataset – IPL(2008-2026)

1.

```
[3] import pandas as pd
import numpy as np

df = pd.read_csv('IPL.csv')

> Show hidden output

[5] #1. Total runs scored per IPL season
seasonal_runs = (
    df.groupby('season')['runs_total']
      .sum()
      .reset_index()
      .rename(columns={'runs_total': 'total_runs'})
)

# Fix season format
seasonal_runs['season'] = seasonal_runs['season'].astype(str).str[:4]
seasonal_runs['season'] = pd.to_numeric(seasonal_runs['season'], errors='coerce')

# Clean
seasonal_runs = seasonal_runs.dropna(subset=['season'])
seasonal_runs['season'] = seasonal_runs['season'].astype(int)

# Final grouping + sorting
seasonal_runs = seasonal_runs.groupby('season', as_index=False)['total_runs'].sum()
seasonal_runs = seasonal_runs.sort_values('season')

print(seasonal_runs)
```

	season	total_runs
0	2007	17937
1	2009	35236
2	2011	21154
3	2012	22453
4	2013	22602
5	2014	18931
6	2015	18353
7	2016	18862
8	2017	18786
9	2018	19901
10	2019	19434
11	2020	19416
12	2021	18637
13	2022	24395
14	2023	25688
15	2024	25971
16	2025	26527
17	2026	8567

Result : Run totals increase over the years as IPL expanded from
~50 to 74 matches per season, with recent seasons crossing
25,000+ total runs. Reflects T20 batting evolution.

Functions: `groupby()`, `sum()`, `reset_index()`, `rename()`, `sort_values()`

2.

```
#2. Top 10 highest run-scorers across all seasons
top_batters = (df.groupby('batter')['runs_batter']
               .sum()
               .reset_index()
               .rename(columns={'runs_batter': 'total_runs'})
               .sort_values('total_runs', ascending=False)
               .head(10))

print(top_batters.to_string(index=False))
```

...	batter	total_runs
	V Kohli	8899
	RG Sharma	7185
	S Dhawan	6769
	DA Warner	6567
	SK Raina	5536
	MS Dhoni	5439
	KL Rahul	5346
	AM Rahane	5184
	AB de Villiers	5181
	CH Gayle	4997

Result : Virat Kohli leads the all-time IPL run charts with 8000+
runs, followed by Shikhar Dhawan and Rohit Sharma.
Consistency over many seasons drives the top positions.

Functions: `groupby()`, `sum()`, `reset_index()`, `rename()`, `sort_values()`, `head()`

3.

```
#3. Most wickets taken by bowlers (all seasons)
# Valid wicket deliveries (exclude run-outs)
wickets = df[df['player_out'].notna() &
              ~df['wicket_kind'].isin(['run out', 'obstructing the field'])]

top_bowlers = (wickets.groupby('bowler')
               .size()
               .reset_index(name='wickets')
               .sort_values('wickets', ascending=False)
               .head(10))

print(top_bowlers.to_string(index=False))
```

bowler	wickets
YS Chahal	224
B Kumar	205
SP Narine	195
PP Chawla	192
R Ashwin	188
JJ Bumrah	186
DJ Bravo	183
A Mishra	174
RA Jadeja	173
SL Malinga	170

Result : Yuzvendra Chahal and B Kumar top the wicket charts.

Run-outs are excluded as they are not credited to the bowler.

Spin bowlers dominate due to consistent economy + wickets.

Functions: notna(), isin(), groupby(), size(), reset_index(), sort_values()

4.

```
#4. Count of matches played at each venue
venue_matches = (df.groupby('venue')['match_id']
                  .nunique()
                  .reset_index(name='matches_played')
                  .sort_values('matches_played', ascending=False))

print(venue_matches.head(10).to_string(index=False))
```

venue	matches_played
Eden Gardens	77
Wankhede Stadium	73
M Chinnaswamy Stadium	65
Feroz Shah Kotla	60
Wankhede Stadium, Mumbai	55
Rajiv Gandhi International Stadium, Uppal	49
MA Chidambaram Stadium, Chepauk	48
Sawai Mansingh Stadium	47
Dubai International Cricket Stadium	46
MA Chidambaram Stadium, Chepauk, Chennai	37

Result : Wankhede (Mumbai) and Eden Gardens(Kolkata) host the most matches. Home-ground advantage plays a key role in team strategies and win rates at these venues.

Functions: groupby(), nunique(), reset_index(), sort_values(), head()

5.

```
#5. Percentage of dot balls bowled per season
valid = df[df['valid_ball'] == 1].copy()
valid['is_dot'] = (valid['runs_total'] == 0).astype(int)

dot_pct = (valid.groupby('season')
            .apply(lambda g: np.round(g['is_dot'].sum() / len(g) * 100, 2))
            .reset_index(name='dot_ball_pct'))

print(dot_pct)
```

```
...      season  dot_ball_pct
0         2009         39.38
1         2011         38.68
2         2012         36.27
3         2013         38.32
4         2014         36.24
5         2015         36.31
6         2016         34.14
7         2017         33.96
8         2018         34.42
9         2019         36.03
10        2021         35.90
11        2022         37.08
12        2023         33.94
13        2024         32.93
14        2025         32.33
15        2026         32.17
16  2007/08         37.96
17        2009         38.66
18  2009/10         35.84
19        2011         37.70
20        2019         35.20
21  2020/21         34.74
22        2021         37.17
```

Result : Around 35–40% of all valid deliveries are dot balls.

Bowlers who consistently bowl dots create pressure;

this metric also tracks how attacking each era of batting was.

Functions: copy(), astype(), groupby(), apply(), np.round(), reset_index()

6.

```
#6. Strike rate of batters (min 500 balls faced)
batter_stats = df.groupby('batter').agg(
    runs=('runs_batter', 'sum'),
    balls=('valid_ball', 'sum')
).reset_index()

# Filter qualified batters
qualified = batter_stats[batter_stats['balls'] >= 500].copy()
qualified['strike_rate'] = np.round(qualified['runs'] / qualified['balls'] * 100, 2)

print(qualified.sort_values('strike_rate', ascending=False).head(10).to_string(index=False))
```

batter	runs	balls	strike_rate
TH David	993	549	180.87
PD Salt	1195	677	176.51
AD Russell	2655	1508	176.06
TM Head	1266	745	169.93
H Klaasen	1704	1020	167.06
SP Narine	1816	1088	166.91
Abhishek Sharma	1945	1168	166.52
N Pooran	2335	1409	165.72
RM Patidar	1333	823	161.97
T Stubbs	820	511	160.47

Result : Tim David and P Salt post strike rates above 175,
far above the IPL average of ~135. Only batters with 500+
balls are considered to ensure statistical significance.

Functions: groupby(), agg(), reset_index(), np.round(), sort_values()

7.

```
#7. Economy rate of bowlers (min 300 balls bowled)
bowler_stats = df.groupby('bowler').agg(
    balls=('valid_ball', 'sum'),
    runs=('runs_bowler', 'sum')
).reset_index()

qualified = bowler_stats[bowler_stats['balls'] >= 300].copy()
qualified['overs'] = qualified['balls'] / 6
qualified['economy'] = np.round(qualified['runs'] / qualified['overs'], 2)

print(qualified.sort_values('economy').head(10).to_string(index=False))
```

...	bowler	balls	runs	overs	economy
	A Kumble	965	1058	160.833333	6.58
	GD McGrath	324	357	54.000000	6.61
	M Muralitharan	1528	1706	254.666667	6.70
	RE van der Merwe	443	498	73.833333	6.74
	DL Vettori	777	879	129.500000	6.79
	SP Narine	4441	5036	740.166667	6.80
	J Yadav	390	445	65.000000	6.85
	J Botha	694	800	115.666667	6.92
	DW Steyn	2182	2523	363.666667	6.94
	AD Mascarenhas	308	356	51.333333	6.94

Result : Legend Anil Kumble consistently concedes under 6.6 runs/over, making him the most economical qualified bowler in IPL.

Economy under 7.0 is considered excellent in T20 cricket.

Functions: groupby(), agg(), reset_index(), np.round(), sort_values()

8.

```
▶ #8. Top 10 six-hitters of all time
print("Top 10 six-hitters of all time")
# Filter only rows where batter hit a six
sixes = df[df['runs_batter'] == 6]

top_six_hitters = (sixes.groupby('batter')
                  .size()
                  .reset_index(name='total_sixes')
                  .sort_values('total_sixes', ascending=False)
                  .head(10))

print(top_six_hitters.to_string(index=False))
print()
```

```
... Top 10 six-hitters of all time
      batter  total_sixes
      CH Gayle          359
      RG Sharma          311
      V Kohli           300
      MS Dhoni           264
AB de Villiers          253
      DA Warner          236
      SV Samson          227
      KA Pollard          224
      AD Russell          223
      KL Rahul           213
```

Result : Chris Gayle leads with 350+ sixes, followed by Rohit Sharma and Virat Kohli. Power hitters from the middle order dominate this list, reflecting the importance of sixes in T20 wins.

Functions: Boolean masking (==), groupby(), size(), reset_index(), sort_values()

9.

```
#9. Which innings (1st or 2nd) scores more runs on average?
print("Average runs scored in 1st innings vs 2nd innings")

innings_runs = (df[(df['valid_ball'] == 1) & (df['innings'].isin([1, 2]))]
                .groupby(['match_id', 'innings'])['runs_total']
                .sum()
                .reset_index())

avg_by_innings = innings_runs.groupby('innings')['runs_total'].mean().round(2)

print(avg_by_innings.to_string())
print(f"\nDifference (1st - 2nd): {np.round(avg_by_innings[1] - avg_by_innings[2], 2)} runs")
print()

... Average runs scored in 1st innings vs 2nd innings
innings
1      161.21
2      148.40

Difference (1st - 2nd): 12.81 runs
```

Result : 1st innings average (~161) is higher than 2nd (~148), showing teams bat more freely without a target. The ~13-run difference highlights the advantage of setting a score.

Functions: `isin()`, `groupby()`, `sum()`, `mean()`, `round()`, `np.round()`

10.

```
#10. Most frequent batting partnerships (50+ runs)
partnerships = (df.groupby(['match_id', 'innings', 'batting_partners'])['runs_total']
                .sum()
                .reset_index(name='partnership_runs'))

big_stands = (partnerships[partnerships['partnership_runs'] >= 50]
              .groupby('batting_partners')
              .agg(count=('partnership_runs', 'count'),
                   avg=('partnership_runs', 'mean'))
              .sort_values('count', ascending=False)
              .head(10))

print(big_stands.round(1).to_string())
```

batting_partners	count	avg
('CH Gayle', 'V Kohli')	21	98.2
('AB de Villiers', 'V Kohli')	21	105.3
('DA Warner', 'S Dhawan')	20	85.1
('Ishan Kishan', 'RG Sharma')	18	70.9
('B Sai Sudharsan', 'Shubman Gill')	16	102.8
('F du Plessis', 'V Kohli')	15	98.9
('CH Gayle', 'KL Rahul')	15	80.9
('KL Rahul', 'MA Agarwal')	15	87.0
('G Gambhir', 'RV Uthappa')	15	86.3
('D Padikkal', 'V Kohli')	13	90.6

Result : Legendary pairs like Kohli- De Villiers and Dhawan–Warner

have built 50+ partnerships the most number of times.

Strong opening partnerships directly correlate with team wins.

Functions: groupby(), sum(), reset_index(), agg(), sort_values(), head()

11.

```
#11. Total extras (wides + no-balls) conceded per team
print("Total extras conceded per bowling team")
extras = df[df['extra_type'].isin(['wides', 'noballs'])].copy()

extras_per_team = (extras.groupby('bowling_team')['runs_extras']
                    .sum()
                    .reset_index(name='total_extras')
                    .sort_values('total_extras', ascending=False))

print(extras_per_team.to_string(index=False))
print()
```

```
*** Total extras conceded per bowling team
      bowling_team  total_extras
0      Mumbai Indians          1627
1  Rajasthan Royals          1393
2  Kolkata Knight Riders          1301
3   Chennai Super Kings          1299
4  Royal Challengers Bangalore          1284
5    Kings XI Punjab           993
6   Sunrisers Hyderabad           902
7   Delhi Daredevils           750
8    Delhi Capitals           586
9    Punjab Kings           444
10  Lucknow Super Giants           388
11  Deccan Chargers           377
12   Gujarat Titans           303
13    Pune Warriors           200
14  Royal Challengers Bengaluru          186
15    Gujarat Lions           114
16  Rising Pune Supergiants           81
```

Result : MI has conceded most extras in total.

Some teams give away 1000+ wides/no-balls per season,
gifting the batting side free runs under pressure.

Functions: `isin()`, `groupby()`, `sum()`, `reset_index()`, `sort_values()`

12.

```
#12. Wicket kinds distribution (caught, bowled, lbw, etc.)
wicket_dist = (df[df['wicket_kind'].notna()]
               ['wicket_kind']
               .value_counts(normalize=True)
               .mul(100)
               .round(2)
               .reset_index())
wicket_dist.columns = ['wicket_kind', 'percentage']

# NumPy cumulative sum for running total
wicket_dist['cumulative'] = np.cumsum(wicket_dist['percentage']).round(2)

print(wicket_dist.to_string(index=False))
```

wicket_kind	percentage	cumulative
caught	62.95	62.95
bowled	16.87	79.82
run out	8.29	88.11
lbw	6.10	94.21
caught and bowled	2.79	97.00
stumped	2.68	99.68
retired hurt	0.13	99.81
hit wicket	0.13	99.94
retired out	0.04	99.98
obstructing the field	0.02	100.00

Result : "Caught" accounts for ~62% of all dismissals in IPL, followed by bowled (~16%) and runout(~8%). Top 3 modes cover ~80% of all wickets — shown via np.cumsum().

Functions: notna(), value_counts(), mul(), round(), np.cumsum()

13.

```
#13. Most runs scored in a single over (across all matches)
# Category: Basic | Rubric: Engineering Knowledge (2 marks)
# =====
print("Most runs scored in a single over (top 10)")
over_runs = (df.groupby(['match_id', 'innings', 'over'])['runs_total']
              .sum()
              .reset_index(name='over_runs')
              .sort_values('over_runs', ascending=False)
              .head(10))

print(over_runs.to_string(index=False))
print(f"\nHighest ever runs in one over: {over_runs['over_runs'].max()}")
print()
```

•• Most runs scored in a single over (top 10)

match_id	innings	over	over_runs
501247	2	2	37
1254076	1	19	37
1304060	2	15	35
734047	2	5	33
419139	1	12	33
1473489	1	18	33
1422138	1	19	32
1473490	2	12	32
548327	2	12	31
1359532	2	15	31

Highest ever runs in one over: 37

Result : Some overs have yielded 30+ runs including wides and no-balls. These explosive overs often feature multiple sixes and fielding errors that change match outcomes.

Highest - 37

Functions: groupby(), sum(), reset_index(), sort_values(), max(), head()

14.

```
#14. Number of matches played per season

print("14. Number of matches played per IPL season")
matches_per_season = df.groupby('season')['match_id'].nunique().reset_index(name='matches_played')

# Use the raw season string as the label – just sort lexicographically by first 4 digits
matches_per_season['season_str'] = matches_per_season['season'].astype(str)
matches_per_season['sort_key'] = matches_per_season['season_str'].str[:4].astype(int)

# Sum duplicate raw-season chunks (same label split across CSV rows)
matches_per_season = (matches_per_season
                      .groupby(['sort_key', 'season_str'])['matches_played']
                      .sum()
                      .reset_index()
                      .sort_values('sort_key')
                      .drop(columns='sort_key'))

print(matches_per_season.to_string(index=False))
print(f"\nTotal matches in dataset: {matches_per_season['matches_played'].sum()}")
print(f"Average matches per season: {np.round(matches_per_season['matches_played'].mean(), 1)}")
print()
```

season_str	matches_played
2007/08	58
2009	59
2009/10	60
2011	74
2012	74
2013	76
2014	60
2015	59
2016	60
2017	59
2018	60
2019	61
2020/21	60
2021	61
2022	74
2023	74
2024	71
2025	74
2026	24

Total matches in dataset: 1198
Average matches per season: 63.1

Result : Early seasons had ~35–60 matches; from 2022 onwards 74

matches per season are played with 10 teams. COVID-affected

2020/21 season was split across UAE and India.

Functions: groupby(), nunique(), astype(), str[:4], sort_values(), sum(), np.round()

15.

```
#15. Over-by-over run rate trend (average across all matches)
over_avg = (df[df['valid_ball']==1]
            .groupby(['match_id','innings','over'])['runs_total']
            .sum()
            .groupby('over')
            .mean()
            .round(3))

# NumPy rolling mean via convolution
window = 3
kernel = np.ones(window) / window
smoothed = np.convolve(over_avg.values, kernel, mode='same')

result = pd.DataFrame({
    'over': over_avg.index,
    'avg_runs': over_avg.values.round(2),
    'smoothed': smoothed.round(2)
})
print(result.to_string(index=False))
```

```
***   over  avg_runs  smoothed
      0      5.91      4.35
      1      7.14      7.03
      2      8.04      7.83
      3      8.32      8.23
      4      8.34      8.34
      5      8.37      7.80
      6      6.69      7.42
      7      7.19      7.14
      8      7.53      7.38
      9      7.43      7.55
     10      7.70      7.66
     11      7.86      7.82
     12      7.89      7.95
     13      8.11      8.14
     14      8.42      8.39
     15      8.64      8.68
     16      8.96      9.04
     17      9.52      9.37
     18      9.63      9.74
     19     10.07      6.57
```

Result : Overs 1–6 (powerplay) average ~7–8 runs; middle overs

Stay consistent at around ~7 ; death overs (16–20) spike to 9 and above.

Functions: groupby(), mean(), round(), reset_index(), rename(), idxmax()

16.

```
#16. Top 10 most used bowlers (most balls bowled)
print("Top 10 most used bowlers by balls bowled")

balls_bowled = (df[df['valid_ball'] == 1]
                .groupby('bowler')['valid_ball']
                .sum()
                .reset_index(name='balls_bowled')
                .sort_values('balls_bowled', ascending=False)
                .head(10))

balls_bowled['overs'] = np.round(balls_bowled['balls_bowled'] / 6, 1)

print(balls_bowled.to_string(index=False))
print()
```

```
... Top 10 most used bowlers by balls bowled
      bowler  balls_bowled  overs
R Ashwin      4710    785.0
SP Narine      4441    740.2
B Kumar        4342    723.7
RA Jadeja      4098    683.0
YS Chahal      3869    644.8
PP Chawla      3850    641.7
JJ Bumrah      3473    578.8
Harbhajan Singh 3416    569.3
AR Patel       3405    567.5
A Mishra       3371    561.8
```

Result : Workhorses like R Ashwin, Sunil Narine and

B Kumar have bowled 4000+ balls across their IPL careers.

np.round() converts balls to overs for readable output.

Functions: groupby(), sum(), reset_index(), sort_values(), np.round(), head()

17.

```
#17. Highest individual score by a batter in a single match

print("Highest individual score by a batter in a single match")

top_scores = (df.groupby(['match_id', 'batter'])['runs_batter']
              .sum()
              .reset_index(name='runs_scored')
              .sort_values('runs_scored', ascending=False)
              .head(10))

print(top_scores.to_string(index=False))
print(f"\nHighest ever individual score: {top_scores['runs_scored'].max()}")
print()
```

```
... Highest individual score by a batter in a single match
  match_id  batter  runs_scored
    598027   CH Gayle          175
    335982   BB McCullum         158
   1473464 Abhishek Sharma         141
   1304112    Q de Kock         140
    829795 AB de Villiers         133
   1216510    KL Rahul          132
    980987 AB de Villiers         129
   1370352  Shubman Gill         129
    548372   CH Gayle          128
   1136602   RR Pant           128

Highest ever individual score: 175
```

Result : Chris Gayle's 175* vs PWI (2013) is the highest IPL score,

followed by Brendon McCullum's 158* in the inaugural match.

These innings single-handedly won their respective matches.

Functions: groupby(), sum(), reset_index(), sort_values(), max(), head()

18.

```
#18. Total number of wides and no-balls per season
print("Total wides and no-balls per season")
wides = df[df['extra_type'] == 'wides'].groupby('season').size().rename('wides')
noballs = df[df['extra_type'] == 'noballs'].groupby('season').size().rename('no_balls')

illegal_deliveries = pd.concat([wides, noballs], axis=1).fillna(0).astype(int).reset_index()
illegal_deliveries['total'] = illegal_deliveries['wides'] + illegal_deliveries['no_balls']

# Keep original season label, sort by first 4 digits, sum duplicate chunks
illegal_deliveries['season_str'] = illegal_deliveries['season'].astype(str)
illegal_deliveries['sort_key'] = illegal_deliveries['season_str'].str[:4].astype(int)
illegal_deliveries = (illegal_deliveries
                      .groupby(['sort_key', 'season_str'])[['wides', 'no_balls', 'total']]
                      .sum()
                      .reset_index()
                      .sort_values('sort_key')
                      .drop(columns='sort_key'))

print(illegal_deliveries.to_string(index=False))
print()
```

Total wides and no-balls per season			
season_str	wides	no_balls	total
2007/08	457	82	539
2009	438	73	511
2009/10	498	62	560
2011	505	83	588
2012	497	62	559
2013	478	62	540
2014	399	55	454
2015	450	52	502
2016	429	49	478
2017	422	39	461
2018	439	39	478
2019	397	58	455
2020/21	453	47	500
2021	461	76	537
2022	702	69	771
2023	626	92	718
2024	729	69	798
2025	699	67	766
2026	258	20	278

Result : Wides outnumber no-balls by 3:1 across all seasons.

Seasons with more matches naturally show more extras.

Recent seasons show better bowling discipline overall.

Functions: groupby(), size(), pd.concat(), fillna(), astype(), str[:4], sort_values()

19.

```
#19. Batter who faced the most balls without scoring
print("Batters who faced the most dot balls")
dot_balls_faced = df[(df['valid_ball'] == 1) & (df['runs_batter'] == 0)]

most_dots = (dot_balls_faced.groupby('batter')
              .size()
              .reset_index(name='dot_balls_faced')
              .sort_values('dot_balls_faced', ascending=False)
              .head(10))

print(most_dots.to_string(index=False))
print()
```

Batters who faced the most dot balls	
batter	dot_balls_faced
V Kohli	2186
RG Sharma	2040
S Dhawan	1970
DA Warner	1713
AM Rahane	1489
RV Uthappa	1464
CH Gayle	1462
SK Raina	1334
MS Dhoni	1327
KL Rahul	1302

Result : Openers like Rohit Sharma, Kohli and

Dhawan top this list due to their high ball volume.

Dot balls faced also indicates a batter's defensive role.

Functions: Boolean masking (&), groupby(), size(), reset_index(), sort_values()

20.

```
#20. Bowlers with most maiden overs (0 runs in full over)

print("Bowlers with most maiden overs (0 runs conceded in full over)")

# A maiden over = full 6 valid balls bowled, 0 runs scored in that over
over_summary = (df[df['valid_ball'] == 1]
                .groupby(['match_id', 'innings', 'bowler', 'over'])
                .agg(balls=('valid_ball', 'sum'), runs=('runs_total', 'sum'))
                .reset_index())

# Keep only complete overs with 0 runs
maidens = over_summary[(over_summary['balls'] == 6) & (over_summary['runs'] == 0)]

maiden_counts = (maidens.groupby('bowler')
                 .size()
                 .reset_index(name='maiden_overs')
                 .sort_values('maiden_overs', ascending=False)
                 .head(10))

print(maiden_counts.to_string(index=False))
```

```
... Bowlers with most maiden overs (0 runs conceded in full over)
      bowler  maiden_overs
      B Kumar             14
      P Kumar             14
      DW Steyn             9
      TA Boult             9
      DS Kulkarni          9
      DL Chahar            8
      IK Pathan            8
      I Sharma             7
      SL Malinga           7
      UT Yadav             7
```

Result : Maiden overs are rare in T20; Economical Bowlers like B Kumar, P Kumar and Dale Steyn achieve them most.

Functions: groupby(), agg(), reset_index(), Boolean masking, size(), sort_values()